

C1A: HTTP Capture File Analysis

Africahackon Training — Wireshark http.cap Assignment

Name: _____ Date: _____

1. Source and destination IP addresses — which is the client and which is the server?

The initial HTTP communication takes place between **145.254.160.237** (source) and **65.208.228.223** (destination). **145.254.160.237 is the client** and **65.208.228.223 is the server**. This can be justified in three ways:

(a) 145.254.160.237 initiates the connection — it sends the very first packet, a TCP SYN, which is always sent by the client in a TCP three-way handshake. (b) 145.254.160.237 connects **to port 80**, the well-known HTTP server port, while using a high, randomly assigned ephemeral port (3372) as its own source port — a classic client pattern. (c) 145.254.160.237 is the host that sends the HTTP GET request (packet 4), and requests are always issued by clients; 65.208.228.223 replies with the HTTP response (200 OK), which is server behaviour.

2. Source and destination port numbers, and the standard HTTP port

The client uses **source port 3372** (an ephemeral/dynamic port chosen by the client's operating system) and the server uses **destination port 80**. On return traffic these are reversed (server source 80 → client destination 3372). The **standard, well-known port for HTTP is TCP port 80** (HTTPS uses 443).

3. Protocol used for connection establishment and the flags in the handshake

The connection between 145.254.160.237 and 65.208.228.223 is established using **TCP (Transmission Control Protocol)** via the three-way handshake in packets 1–3:

Packet	Flags	Meaning
1	[SYN] Seq=0	Client → Server: requests a connection and announces its initial sequence number (synchronise).
2	[SYN, ACK] Seq=0 Ack=1	Server → Client: accepts the connection, acknowledges the client's SYN, and announces its own initial sequence number.
3	[ACK] Seq=1 Ack=1	Client → Server: acknowledges the server's SYN. The connection is now ESTABLISHED and data can flow.

The SYN (synchronise) flag is used to synchronise the initial sequence numbers on both sides, and the ACK (acknowledge) flag confirms receipt of the other side's sequence number. Only after all three steps do both hosts agree on sequence numbering and can reliably exchange data.

4. HTTP method used and the resource requested

In packet 4 the client uses the **GET** method. The request line is `GET /download.html HTTP/1.1` with the header `Host: www.ethereal.com`, so the client is requesting the resource `/download.html` (the downloads page) from the web server **www.ethereal.com**.

5. Two application layer protocols present in the capture

The capture contains at least two application layer protocols: **HTTP** (the web request/response between the client and the web servers) and **DNS** (the query and response used to resolve `pagead2.googlesyndication.com` to an IP address, seen in packets 13 and 17).

6. TCP/IP layer responsible for the reliable connection establishment (packets 1-3)

The three-way handshake is handled by the **Transport layer** of the TCP/IP model (equivalent to Layer 4 of the OSI model). TCP, a transport layer protocol, is responsible for connection establishment, sequencing, acknowledgements, retransmission and flow control — all the mechanisms that make communication reliable.

7. Purpose of the ACK flag and its relation to reliable data transfer

The ACK flag indicates that the **Acknowledgment Number field is valid**: the sender of the packet is confirming receipt of data from the other side. The acknowledgment number states the **next byte the receiver expects**, which implicitly confirms that every byte before it arrived intact. This is the foundation of TCP's reliability: if a sender does not receive an ACK for transmitted data within a timeout period, it assumes the data (or the ACK) was lost and **retransmits** it. ACKs therefore give the sender positive confirmation of delivery, allow lost segments to be detected and recovered, and (together with the window field) regulate how much unacknowledged data may be in flight.

8. Encapsulation of the HTTP GET request (packet 4) from client to server

The GET request is plain application data (ASCII text). As it travels down the client's protocol stack, each layer wraps it with its own header:

Application layer: the browser produces the HTTP message (`GET /download.html HTTP/1.1` plus headers).

Transport layer (TCP): a **TCP header** is prepended, containing the source port (3372), destination port (80), sequence number, acknowledgment number, flags (PSH, ACK) and window size. HTTP data + TCP header = a **segment**.

Internet layer (IP): an **IP header** is added, containing the source IP (145.254.160.237), destination IP (65.208.228.223), TTL, and protocol field (6 = TCP). Segment + IP header = a **packet**.

Network access / link layer: an Ethernet header (source/destination MAC addresses, EtherType) and trailer (FCS checksum) are added, forming a **frame** that is transmitted as bits on the physical medium. Routers along the path use the IP header to forward the packet to the server.

9. Encapsulation of the server's HTTP response and decapsulation by the client

The server performs the same process in reverse direction. The web server software generates the HTTP response (`HTTP/1.1 200 OK` plus headers and the HTML body of `download.html`). TCP adds a header

with source port 80, destination port 3372 and the correct sequence/acknowledgment numbers (the body is large, so it is split across several TCP segments). IP adds a header with source 65.208.228.223 and destination 145.254.160.237, and the link layer frames each packet for transmission.

When the frames arrive at the client, **decapsulation** occurs layer by layer: the NIC verifies the Ethernet frame check sequence and strips the frame header; the IP layer checks the destination address and strips the IP header; TCP uses the destination port (3372) to hand the payload to the correct application socket, uses the sequence numbers to **reassemble the segments in the correct order** and sends ACKs back; finally the browser (application layer) receives the complete HTTP response, reads the status line and headers, and renders the HTML page.

10. The DNS query — domain, record type, and the purpose of DNS

In packet 13 the client queries the domain **pagead2.googlesyndication.com** and requests an **“A” record** — an address record that maps a hostname to an IPv4 address. The response (packet 17) supplies the IP address of the ad server so the client can open a TCP connection to it.

The purpose of DNS (Domain Name System) is to act as the internet's “phone book”: it translates human-readable names (www.example.com) into the numeric IP addresses that the IP layer needs to route packets. Without DNS, users would have to memorise IP addresses, and services could not move between servers transparently.

11. How sequence and acknowledgment numbers ensure reliable data transfer (packets 3-5)

Every byte TCP sends is numbered. The **sequence number** identifies the position of the first byte of a segment in the byte stream, and the **acknowledgment number** tells the peer the next byte number expected — confirming all earlier bytes arrived. This lets the receiver detect missing or duplicate segments and reorder out-of-order arrivals, and lets the sender know exactly what to retransmit if an ACK never comes.

Trace of packets 3-5 (relative numbers):

Packet	Direction	Seq / Ack / Len	Interpretation
3	Client → Server	Seq=1, Ack=1, Len=0	Handshake ACK; both sides start their data streams at byte 1.
4	Client → Server	Seq=1, Ack=1, Len=479	The GET request: 479 bytes, occupying bytes 1–479 of the client's stream.
5	Server → Client	Seq=1, Ack=480, Len=0	Server acknowledges: “I received bytes up to 479; send byte 480 next.”

The server's Ack=480 is exactly the client's Seq (1) plus the payload length (479), proving the entire request arrived. If packet 4 had been lost, no such ACK would arrive and the client would retransmit it.

12. Meaning of the [SYN], [ACK], [SYN, ACK] and [PSH, ACK] flags

Flag(s)	State / purpose of the packet
[SYN]	Connection request. Sent by the client to open a connection and synchronise its initial sequence number. First step of the handshake.
[SYN, ACK]	Connection accepted. The server both acknowledges the client's SYN and sends its own SYN. Second step of the handshake.
[ACK]	Acknowledgment. Confirms receipt of data or of a SYN. Present in almost every packet of an established connection.
[PSH, ACK]	Data delivery. Carries application data; PSH (push) tells the receiving TCP stack to hand the data to the application immediately instead of buffering it. Used for the HTTP GET and response segments.

13. Purpose of the communication with pagead2.googlesyndication.com (packets 13, 17, 18)

pagead2.googlesyndication.com is Google's **advertising (AdSense) content server**. The HTML page returned by www.ethereal.com contains an embedded advertisement, so the browser must fetch that ad from Google. Packet 13 is the DNS A-record query for the ad server's name, packet 17 is the DNS response containing its IP address, and packet 18 is a new HTTP GET request to that server (a URL under /pagead/) to download the ad content/script. This illustrates that loading one web page routinely triggers additional DNS lookups and HTTP connections to third-party servers.

14. Packet 36: [TCP Spurious Retransmission] — what happened and why?

A **spurious retransmission** means the sender retransmitted a segment that the receiver had, in fact, **already received and acknowledged** — Wireshark flags it because it can see the earlier copy and the ACK covering it in the capture. The retransmission was therefore unnecessary.

It suggests the sender **wrongly concluded the segment was lost**. Common causes are: the acknowledgment for the segment was **lost or delayed** in the network, so the sender's retransmission timer (RTO) expired before the ACK arrived; sudden network congestion/latency making the round-trip time exceed the sender's timeout estimate; or duplicate ACKs triggering fast retransmit even though the original data eventually arrived. In short, the data was fine — it was the feedback path (the ACKs) or the sender's timing assumptions that failed. Spurious retransmissions waste bandwidth and may cause TCP to slow down unnecessarily, but the receiver simply discards the duplicate using the sequence numbers, so correctness is preserved.